



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Monterrey, Escuela de Ingeniería y Ciencias, IDM

MA2006B — Uso de Álgebras Modernas para Seguridad y Criptografía

Documentación de Desarrollo

Instar: Gestor de Identidades, Registros y Documentos

Socio Formador

Casa Monarca — Ayuda Humanitaria al Migrante, A.B.P.

Reto: Firmas solidarias y protección de datos personales

Equipo: Equipo 4 **Grupo:** Grupo 602

Versión del sistema: v1.0.0

Profesores: Alberto F. Martínez, Daniel Otero Fadul, Raúl Gómez Muñoz, Pedro Leonel Olaya Trejos

Integrantes del equipo

Juan David Pastrana Arango, A01751471, Líder de desarrollo

Yamil Elías Del Blanco Chávez, A00838610, Líder técnico

Luis Roberto Campos Solís, A00838686, Líder de equipo

Mariano Vertiz Sánchez, A00840430, Desarrollador

Álvaro Duhart Pérez, A01029468, Desarrollador

Mauro Artemio Sotelo Dávila, A01707689, Desarrollador

Monterrey, Nuevo León · 14 de junio de 2026

Índice de contenidos

Índice de contenidos	2
1. Descripción general	5
1.1. Propósito del sistema	5
1.2. Alcance	5
1.3. Casos de uso principales	5
2. Arquitectura del sistema	6
2.1. Tipo de arquitectura	6
2.2. Componentes principales	6
2.3. Diagrama general	7
2.4. Tres flujos de documento (v1.1.0)	8
2.4.1. Flujo 1 — Documentos de expediente (personal → migrante)	8
2.4.2. Flujo 2 — Documentos de identidad (migrante sube y sella)	8
2.4.3. Flujo 3 — Corrección ARCO (migrante solicita rectificación)	8
2.5. Capa criptográfica	8
3. Stack tecnológico	9
3.1. Lenguajes	9
3.2. Frameworks y herramientas	9
3.3. Bibliotecas — composer.json	10
3.4. Bibliotecas — package.json	11
4. Estructura del proyecto	11
4.1. Árbol de carpetas	11
4.2. Descripción de módulos clave (heredados de v1.0.0)	12
4.3. Módulos nuevos o modificados en v1.1.0	13
5. Configuración del entorno	14
5.1. Requisitos	14
5.2. Variables de entorno	14
5.3. Variables de entorno relevantes (resumen)	15
5.4. Archivos de configuración	15
6. Instalación y ejecución	16
6.1. Setup inicial (desarrollo)	16
6.2. Setup con Docker (producción)	16
6.3. Comandos principales	17
7. Base de datos	18
7.1. Modelo de datos	18
7.2. Tablas principales	18
7.3. Columnas nuevas en v1.1.0 sobre la tabla documentos	19
7.4. Migraciones nuevas en v1.1.0	19
7.5. Decisión: bitácora sin llave foránea	20
8. API / Interfaces	20
8.1. Rutas principales (Etapa 1)	20

8.2. Rutas nuevas en v1.1.0	21
8.3. Ejemplo de petición/respuesta — firma challenge-response	23
9. Flujos y lógica de negocio	24
9.1. Emisión de certificado al aprobar Coordinador	24
9.2. Verificación PEM para acciones sensibles (Etapa 1)	25
9.3. Cálculo de hash de documento al subir	26
9.4. Generación de folio CM-AAAA-XXXX	26
9.5. Ciclo de vida de una identidad	27
9.6. Firma digital challenge-response (v1.1.0)	27
9.7. Sellado HMAC-SHA256 de documento de identidad (v1.1.0)	28
9.8. Máquina de estados del flujo ARCO de Rectificación (v1.1.0)	29
10. Ejemplos de uso	29
10.1. Ejecutar pruebas	29
10.2. Crear usuario administrador desde Tinker	29
10.3. Consultar bitácora	30
10.4. Verificar integridad de un documento desde código	30
10.5. Aprobar identidad desde la interfaz	30
10.6. Crear caso directo desde Coordinador (v1.1.0)	31
11. Manejo de errores	31
11.1. Estrategia general	31
11.2. Mensajes comunes	31
11.3. Registro (logs)	32
12. Pruebas	32
12.1. Tipos de tests	32
12.2. Cómo ejecutarlos	32
12.3. Archivos de test nuevos en v1.1.0	33
13. Despliegue	33
13.1. Cómo hacerlo — Docker (recomendado en v1.1.0)	33
13.2. Cómo hacerlo — hosting tradicional	34
13.3. Entornos	34
13.4. Consideraciones	34
14. Seguridad	35
14.1. Autenticación	35
14.2. Manejo de datos sensibles	35
14.3. Buenas prácticas aplicadas	35
15. Guía de contribución	36
15.1. Convenciones de código	36
15.2. Flujo de trabajo (Git)	36
15.3. Cómo agregar funcionalidades	36
15.4. Reglas especiales para código criptográfico (v1.1.0)	37
16. Problemas conocidos	37
16.1. Bugs	37
16.2. Limitaciones	37

17. FAQ técnica	38
17.1. composer install falla en producción	38
17.2. openssl_pkey_new() retorna false	38
17.3. Vite 8 no compila	38
17.4. Tests fallan con error de base de datos	38
17.5. La firma del Coordinador siempre devuelve HTTP 403	38
17.6. selladoEsValido() devuelve false aunque el archivo no haya cambiado	39
17.7. Cómo regenerar el certificado de un coordinador	39
17.8. Cómo descargar dependencias en hostings restringidos	39
17.9. ¿Cómo verifico manualmente la integridad de un documento de identidad?	39
Referencias	40

1. Descripción general

1.1. Propósito del sistema

El sistema Instar — Gestor de Identidades, Registros y Documentos en su versión 1.1.0 centraliza la administración de identidades digitales del personal de Casa Monarca, emite certificados RSA-2048 internos para roles de coordinación, implementa firma digital genuina de documentos de expediente mediante un protocolo challenge-response, sella la integridad de los documentos de identidad de personas migrantes con HMAC-SHA256, opera el flujo completo del Derecho ARCO de Rectificación y mantiene una bitácora inmutable de auditoría. La aplicación está alineada con la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y con los Derechos ARCO (Cámara de Diputados, 2025).

1.2. Alcance

La versión 1.1.0 cubre, como base heredada de la Etapa 1: el ciclo de vida completo de las identidades (alta, suspensión, revocación, baja, restauración); la emisión y revocación de certificados internos; la gestión de áreas, solicitudes, postulaciones, expedientes y documentos; la verificación criptográfica de posesión de llave PEM para acciones sensibles; y la auditoría inmutable. La Etapa 2 añade sobre esa base: firma digital challenge-response RSA-2048 de documentos de expediente; sellado HMAC-SHA256 de documentos de identidad con verificación posterior desde disco; flujo ARCO de Rectificación con máquina de estados de cuatro etapas; visibilidad en dos fases para documentos de expediente; rol Voluntario (nivel 4); vista administrativa de auditoría de archivos por migrante; logging de descargas de staff; creación directa de casos por Coordinador; migración de base de datos de desarrollo a SQLite; y actualización a Vite 8.

Quedan fuera del alcance al cierre de la versión 1.1.0: sistema de correos cifrados (Sub-problema 2); verificación externa pública de documentos por terceros (Sub-problema 4); cifrado de base de datos en reposo (Sub-problema 6); respaldos automáticos cifrados (Sub-problema 7); autenticación multifactor TOTP; y la capa blockchain. La arquitectura está diseñada para que estas extensiones se incorporen sin reescribir los módulos existentes.

1.3. Casos de uso principales

- Alta y aprobación de identidades del personal interno y de personas migrantes.
- Emisión automática de certificado RSA-2048 al aprobar a un Coordinador, con entrega única de llave privada PEM.
- Firma digital de documentos de expediente por parte del Coordinador, desde el navegador, sin que la llave privada salga del cliente.

- Sellado HMAC-SHA256 de documentos de identidad cargados por personas migrantes, con verificación posterior desde la vista administrativa.
- Flujo completo ARCO de Rectificación: solicitud por el migrante, toma por el Operativo, propuesta de corrección, aprobación firmada por el Coordinador.
- Verificación criptográfica de posesión de llave privada para acciones sensibles (edición y eliminación de documentos).
- Registro de solicitudes de servicio por parte de migrantes y postulación del personal operativo a esas solicitudes.
- Apertura de expedientes con folio único CM-AAAA-XXXX.
- Carga de documentos con cálculo automático de huella SHA-256 e integridad documental verificable.
- Revocación y restauración de identidades con marcado automático de certificados activos como revocados.
- Auditoría granular de archivos por persona migrante con verificación de integridad desde disco en tiempo real.
- Creación directa de casos por el Coordinador para flujos de emergencia (sin necesidad de solicitud previa del migrante).

2. Arquitectura del sistema

2.1. Tipo de arquitectura

El sistema se construye como una aplicación web monolítica basada en el patrón Model-View-Controller (MVC) propio de Laravel 13. Las peticiones HTTP son enrutadas a controladores específicos, los controladores delegan la lógica de negocio en servicios dedicados (especialmente para la capa criptográfica), los modelos Eloquent encapsulan el acceso a la base de datos relacional, y las vistas Blade renderizan HTML server-side complementado con Tailwind CSS y Alpine.js en el cliente.

La versión 1.1.0 mantiene esta arquitectura monolítica e introduce una novedad relevante en la capa cliente: el componente de firma digital ejecutado por Alpine.js utiliza la Web Crypto API nativa del navegador para realizar la firma RSASSA-PKCS1-v1_5 con digest SHA-256 sobre un challenge generado por el servidor, sin necesidad de bibliotecas externas y sin que la llave privada salga del cliente.

2.2. Componentes principales

- **Capa de rutas (routes/web.php):** mapea URLs a controladores y aplica middlewares de autenticación y autorización.

- **Controladores HTTP (app/Http/Controllers):** coordinan entrada/salida y delegan en servicios. En v1.1.0 se añaden FirmaController, DocumentoidentidadController, RectificacionController y ArchivosMigrantesController.
- **Servicios de dominio (app/Services):** encapsulan operaciones criptográficas (CertificadoService), generación de folios (FolioService) y hashing (HashService).
- **Modelos Eloquent (app/Models):** User, Certificado, Area, Solicitud, Postulacion, Expediente, Documento, ActividadLog (Etapa 1); SolicitudRectificacion y Firma se añaden en v1.1.0.
- **Políticas y Gates (app/Policies, AppServiceProvider):** autorización por recurso y por rol; en v1.1.0 se añade el Gate firmar-documento.
- **Middlewares (app/Http/Middleware):** CheckStatus, autenticación Breeze, verificación de área.
- **Migraciones (database/migrations):** versionado del esquema relacional. v1.1.0 introduce migraciones para sello_integridad, visible_migrante, firmado_at, solicitudes_rectificacion y el rol Voluntario.
- **Vistas Blade (resources/views):** pantallas agrupadas por dominio (auth, admin, coordinador, operativo, migrante, components).
- **Componente Alpine.js de firma (v1.1.0):** lee archivo .pem por drag-and-drop, ejecuta importKey y sign con Web Crypto API, envía la firma al servidor.

2.3. Diagrama general

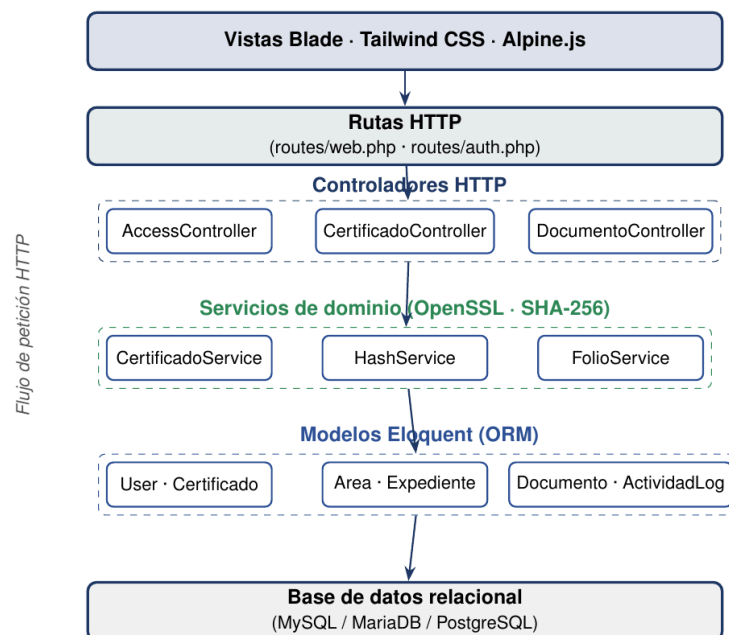


Fig. 1. Arquitectura MVC del sistema en Laravel (elaboración propia).

2.4. Tres flujos de documento (v1.1.0)

La versión 1.1.0 consolida tres flujos claramente diferenciados según el tipo de documento. Esta separación es fundamental para entender cómo navegar la base de código.

2.4.1. Flujo 1 — Documentos de expediente (personal → migrante)

El Operativo sube un PDF al expediente. El archivo se almacena en `/storage/expedientes/{id}/` con `visible_migrante = false`. El Coordinador lo ve en su vista con fondo ámbar y etiqueta "Pendiente de firma". Carga su archivo `.pem` en el componente `Alpine.js` de firma, que ejecuta el protocolo `challenge-response` RSA-2048 (ver sección 9.6). Cuando la firma se verifica como válida, la firma queda guardada en la tabla `firmas`, `visible_migrante` se cambia a `true` y el migrante puede descargar el documento desde su portal.

Controladores principales: `CasoController` (`subirDocumento`, `editarDocumento`, `eliminarDocumento`) y `FirmaController` (`challenge`, `store`).

2.4.2. Flujo 2 — Documentos de identidad (migrante sube y sella)

La persona migrante sube su PDF de identidad (pasaporte, identificación) en `/mi-espacio/documentos`. El sistema calcula `hash_sha256` del contenido binario y lo guarda. Se presenta un modal con el hash visible para el migrante y un botón "Confirmar y sellar". Al confirmar, el sistema calcula `sello_integridad = HMAC_SHA256(hash, app.key)` y persiste `sellado_at = now()`. El Administrador puede verificar en cualquier momento la integridad desde `/admin/archivos` recalculando el hash desde disco (ver sección 9.7).

Controlador principal: `DocumentoidentidadController` (`index`, `store`, `sellar`, `download`, `destroy`).

2.4.3. Flujo 3 — Corrección ARCO (migrante solicita rectificación)

La persona migrante describe la corrección que requiere; se crea un registro `SolicitudRectificacion` con `status = pendiente_correccion`. Un Operativo toma la solicitud (`status = en_proceso`), sube el documento corregido (`status = esperando_aprobacion`), y finalmente el Coordinador revisa y firma con su `.pem` mediante el mismo flujo `challenge-response`. Al aprobarse: el documento original es eliminado lógicamente, el corregido queda activo, y ambas versiones quedan registradas en `actividad_log` con el `fingerprint` del certificado que autorizó el cambio (ver sección 9.8).

Controlador principal: `RectificacionController` (`solicitar`, `tomar`, `subirPropuesta`, `aprobar`).

2.5. Capa criptográfica

La capa criptográfica se distribuye entre servidor (PHP/OpenSSL) y cliente (Web Crypto API). El servidor mantiene la responsabilidad de generación de pares RSA, almacenamiento de llaves públicas,

verificación de firmas y cálculo de HMAC. El cliente añade, en v1.1.0, la responsabilidad de mantener la llave privada del Coordinador y ejecutar la firma sin transferirla al servidor.

- **RSA-2048 PKI server-side (Etapa 1):** `openssl_pkey_new()` en `CertificadoService::emitir()` al aprobar un Coordinador. La llave pública se persiste con fingerprint SHA-256. La privada se entrega una sola vez en sesión flash.
- **Verificación PEM (Etapa 1):** `hash_equals()` para comparar fingerprints en tiempo constante y prevenir timing attacks. Aplica a edición y eliminación de documentos.
- **Firma digital challenge-response (v1.1.0):** el servidor genera un nonce, lo asocia a la sesión y al documento. El cliente firma con RSASSA-PKCS1-v1_5 + SHA-256 vía Web Crypto API. El servidor verifica con `openssl_verify()`.
- **HMAC-SHA256 (v1.1.0):** `hash_hmac('sha256', hash_archivo, config('app.key'))` para sellar documentos de identidad. La `APP_KEY` es la llave HMAC.

3. Stack tecnológico

3.1. Lenguajes

Lenguaje	Uso
PHP 8.3	Backend principal con Laravel; <code>openssl_*</code> para RSA, SHA-256 y HMAC
Blade	Plantillas server-side rendered
JavaScript (ES2020+)	Interactividad cliente con Alpine.js y Web Crypto API (firma)
CSS / Tailwind	Estilos utility-first
SQL	Consultas relacionales gestionadas por Eloquent

3.2. Frameworks y herramientas

Tecnología	Versión	Uso	Cambio v1.1.0
Laravel	13.x	Framework MVC principal	Sin cambio
Laravel Breeze	2.3	Autenticación base	Sin cambio

Tecnología	Versión	Uso	Cambio v1.1.0
Eloquent ORM	incluido	Modelado de datos	Sin cambio
Tailwind CSS	3.1+	Sistema de estilos	Sin cambio
Alpine.js	3.4+	Reactividad cliente; componente de firma	Sin cambio
Vite	8.x	Compilación de assets	Actualizado de 7.0+
Web Crypto API	estándar W3C	Firma RSA-2048 client-side	Nuevo en v1.1.0
OpenSSL PHP	nativa	Generación y validación RSA + HMAC	Sin cambio
SQLite	3.x	Base de datos de desarrollo	Nuevo en v1.1.0 (era MySQL)
MySQL	8.x	Base de datos de producción	Ahora vía Docker
Docker Compose	—	Orquestación de producción	Nuevo en v1.1.0
Composer	2.5+	Gestor de dependencias PHP	Sin cambio
NPM	10+	Gestor de dependencias frontend	Sin cambio
Pest / PHPUnit	Pest 4.4	Framework de pruebas	Actualizado de 4.1
Laravel Pint	1.24	Formateo de código	Sin cambio

Tabla 1. Stack tecnológico v1.1.0 (elaboración propia).

3.3. Bibliotecas — composer.json

```
{
  "require": {
    "php": "^8.3",
    "laravel/framework": "^13.0",
    "laravel/tinker": "^2.10.1"
  },
  "require-dev": {
    "fakerphp/faker": "^1.24",
    "laravel/breeze": "^2.3",
    "laravel/pint": "^1.24",

```

```

    "laravel/sail": "^1.41",
    "nunomaduro/collision": "^8.8",
    "pestphp/pest": "^4.4",
    "pestphp/pest-plugin-laravel": "^4.0"
  }
}

```

3.4. Bibliotecas — package.json

```

{
  "devDependencies": {
    "@tailwindcss/forms": "^0.5.2",
    "@tailwindcss/vite": "^4.0.0",
    "alpinejs": "^3.4.2",
    "axios": "^1.11.0",
    "laravel-vite-plugin": "^2.0.0",
    "tailwindcss": "^3.1.0",
    "vite": "^8.0.0"
  }
}

```

4. Estructura del proyecto

4.1. Árbol de carpetas

```

casa-monarca/
├── app/
│   ├── Http/
│   │   ├── Controllers/ (controladores HTTP del dominio)
│   │   │   ├── UserController.php
│   │   │   ├── CertificadoController.php
│   │   │   ├── AccessController.php
│   │   │   ├── SolicitudController.php
│   │   │   ├── ExpedienteController.php
│   │   │   ├── CasoController.php
│   │   │   ├── DocumentoController.php
│   │   │   ├── FirmaController.php [v1.1.0]
│   │   │   ├── DocumentoIdentidadController.php [v1.1.0]
│   │   │   ├── RectificacionController.php [v1.1.0]
│   │   │   ├── ArchivosMigrantesController.php [v1.1.0]
│   │   │   └── CoordinadorController.php
│   │   ├── Middleware/ (CheckStatus, auth, area)
│   │   └── Requests/ (FormRequests de validación)
│   ├── Models/
│   │   ├── User.php · Certificado.php · Area.php
│   │   ├── Solicitud.php · Postulacion.php
│   │   ├── Expediente.php · Documento.php
│   │   ├── ActividadLog.php
│   │   ├── SolicitudRectificacion.php [v1.1.0]
│   │   └── Firma.php [v1.1.0]
│   ├── Policies/ (autorización por recurso)
│   ├── Services/ (CertificadoService, HashService, FolioService)
│   └── Providers/ (AppServiceProvider – Gates)
├── bootstrap/
└── config/

```

```

├── database/
│   ├── factories/
│   ├── migrations/           (esquema versionado)
│   ├── seeders/             (datos iniciales)
│   └── database.sqlite       (base de datos local – no versionada) [v1.1.0]
├── public/
├── resources/
│   ├── css/
│   ├── js/
│   │   └── components/firma.js (componente Alpine.js de firma) [v1.1.0]
│   └── views/                (plantillas Blade)
│       ├── auth/ · admin/ · coordinador/ · operativo/ · migrante/
│       └── components/
├── routes/
│   ├── web.php               (rutas principales)
│   └── auth.php              (rutas de Breeze)
├── storage/
│   ├── expedientes/         (documentos de expediente)
│   ├── identidad/           (documentos de identidad de migrantes) [v1.1.0]
│   └── logs/                 (laravel.log con rotación diaria)
├── tests/
│   ├── Feature/              (Pest – tests end-to-end)
│   └── Unit/
├── compose.yaml               (Docker Compose para producción) [v1.1.0]
├── .env.example
├── composer.json · package.json · vite.config.js
├── CLAUDE.md                  (guía para desarrollo asistido por IA) [v1.1.0]
├── README.md · TODO.md · CHANGELOG.md · LICENSE

```

4.2. Descripción de módulos clave (heredados de v1.0.0)

Módulo	Responsabilidad
app/Http/Controllers/AccessController	Bandeja de Accesos del Administrador
app/Http/Controllers/CertificadoController	Emisión, consulta y revocación de certificados
app/Http/Controllers/SolicitudController	Ciclo de vida de solicitudes de migrantes
app/Http/Controllers/ExpedienteController	Apertura y gestión de expedientes
app/Http/Controllers/DocumentoController	Carga, edición y eliminación de documentos
app/Services/CertificadoService	openssl_pkey_new, fingerprint, validación PEM
app/Services/HashService	Cálculo de hash SHA-256 de archivos
app/Services/FolioService	Generación de folios CM-AAAA-XXXX

Módulo	Responsabilidad
app/Http/Middleware/CheckStatus	Bloqueo de cuentas no activas

4.3. Módulos nuevos o modificados en v1.1.0

Módulo	Responsabilidad
app/Http/Controllers/FirmaController.php	challenge() genera nonce; store() verifica firma RSA y activa visible_migrante
app/Http/Controllers/DocumentoIdentidadController.php	store() calcula SHA-256; sellar() aplica HMAC; download() registra en log; destroy()
app/Http/Controllers/RectificacionController.php	solicitar(), tomar(), subirPropuesta(), aprobar() — máquina de estados ARCO
app/Http/Controllers/ArchivosMigrantesController.php	Galería /admin/archivos, detalle por migrante, verificación de integridad desde disco
app/Models/Documento.php	selladoEsValido() — verifica HMAC; visible_migrante (bool); relaciones con Firma
app/Models/SolicitudRectificacion.php	Status enum + relaciones + scopes para filtrar por estado
app/Models/Firma.php	doc_id, coordinador_id, firma_base64, firmado_at
app/Providers/AppServiceProvider.php	Definiciones de Gates de autorización (firmar-documento)
database/migrations/2026_06_04_000001_*.php	Añade sello_integridad (TEXT) y sellado_at (TIMESTAMP) a tabla documentos
resources/js/components/firma.js	Componente Alpine.js con drag-and-drop de .pem y firma Web Crypto API

Tabla 2. Módulos nuevos o modificados en v1.1.0 (elaboración propia).

5. Configuración del entorno

5.1. Requisitos

- PHP 8.3 o superior con extensiones: openssl, pdo, pdo_sqlite (v1.1.0), pdo_mysql, mbstring, fileinfo, bcmath, ctype, json.
- Composer 2.5+.
- Node.js 18+ y NPM 10+.
- SQLite (desarrollo) — incluido en la mayoría de sistemas; verificar con `php -m | grep sqlite`.
- MySQL 8 (producción), opcionalmente orquestado vía Docker.
- Docker + Docker Compose (opcional, recomendado para despliegue de producción).
- Git para clonar el repositorio.

5.2. Variables de entorno

El archivo `.env` contiene la configuración del ambiente. Las variables relevantes son:

```
APP_NAME="Casa Monarca"
APP_ENV=local                # cambiar a production antes de go-live
APP_KEY=base64:...          # generada con php artisan key:generate
APP_DEBUG=true              # cambiar a false en producción
APP_URL=http://localhost:8000

# Desarrollo (v1.1.0) - SQLite
DB_CONNECTION=sqlite
DB_DATABASE=database/database.sqlite

# Producción - MySQL (vía Docker)
# DB_CONNECTION=mysql
# DB_HOST=db                 # nombre del servicio en compose.yaml
# DB_PORT=3306
# DB_DATABASE=casa_monarca
# DB_USERNAME=...
# DB_PASSWORD=...

SESSION_DRIVER=database
SESSION_LIFETIME=120

MAIL_MAILER=log             # cambiar a smtp en producción

# APP_KEY se usa también como llave HMAC para sello_integridad de
# documentos de identidad. No rotar sin re-sellar.
```

5.3. Variables de entorno relevantes (resumen)

Variable	Valor dev	Descripción
APP_KEY	base64:...	Generada con artisan key:generate. Usada también como llave HMAC.
APP_ENV	local	Cambiar a production antes de go-live.
APP_DEBUG	true	Cambiar a false en producción.
DB_CONNECTION	sqlite	sqlite para dev, mysql para producción.
DB_DATABASE	database/database.sqlite	Ruta para SQLite o nombre de BD para MySQL.
MAIL_DRIVER	log	Sin correos reales en dev; revisar storage/logs/laravel.log.

Tabla 3. Variables de entorno (elaboración propia).

5.4. Archivos de configuración

- config/app.php — configuración general.
- config/database.php — conexiones de base de datos (sqlite y mysql).
- config/auth.php — guards y providers.
- config/session.php — driver y duración de sesiones.
- config/filesystems.php — discos de almacenamiento (expedientes, identidad, public).

6. Instalación y ejecución

6.1. Setup inicial (desarrollo)

```
# 1. Clonar el repositorio
git clone https://github.com/A01751471/casa-monarca.git
cd casa-monarca

# 2. Instalar dependencias PHP
composer install

# 3. Instalar dependencias frontend
npm install

# 4. Copiar archivo de entorno
cp .env.example .env

# 5. Generar clave de aplicación (también será llave HMAC)
php artisan key:generate

# 6. Crear la base de datos SQLite local (v1.1.0)
touch database/database.sqlite

# 7. Ejecutar migraciones y seeders
php artisan migrate --seed

# 8. Compilar assets
npm run build

# 9. Levantar el servidor
php artisan serve

# Alternativa todo-en-uno (v1.1.0): servidor + vite + queue + logs
composer run dev
```

6.2. Setup con Docker (producción)

1. Copiar `.env.example` a `.env` y configurar `DB_CONNECTION=mysql`, credenciales y `APP_DEBUG=false`.
2. Levantar los servicios: `docker compose up -d`
3. Ejecutar migraciones dentro del contenedor: `docker compose exec app php artisan migrate --seed`
4. Compilar assets dentro del contenedor: `docker compose exec app npm run build`

El archivo `compose.yaml` define tres servicios: `app` (PHP 8.3 + Laravel), `db` (MySQL 8) y `redis` (cache y sesiones). El servicio `app` expone el puerto 8000; `db` y `redis` quedan internos.

6.3. Comandos principales

Comando	Propósito
php artisan serve	Servidor de desarrollo en http://127.0.0.1:8000
npm run dev	Compila assets en modo desarrollo (watch)
npm run build	Compila assets para producción
php artisan migrate	Aplica migraciones pendientes
php artisan migrate:fresh --seed	Reinicia la base con datos semilla (¡destructivo!)
php artisan test	Ejecuta toda la suite de pruebas
./vendor/bin/pest	Ejecuta tests con Pest
./vendor/bin/pint	Formatea código según convenciones Laravel
php artisan tinker	REPL interactivo para inspección
composer run dev	Levanta servidor + Vite + queue + logs concurrentemente (v1.1.0)
docker compose up -d	Levanta el stack completo en producción (v1.1.0)

7. Base de datos

7.1. Modelo de datos

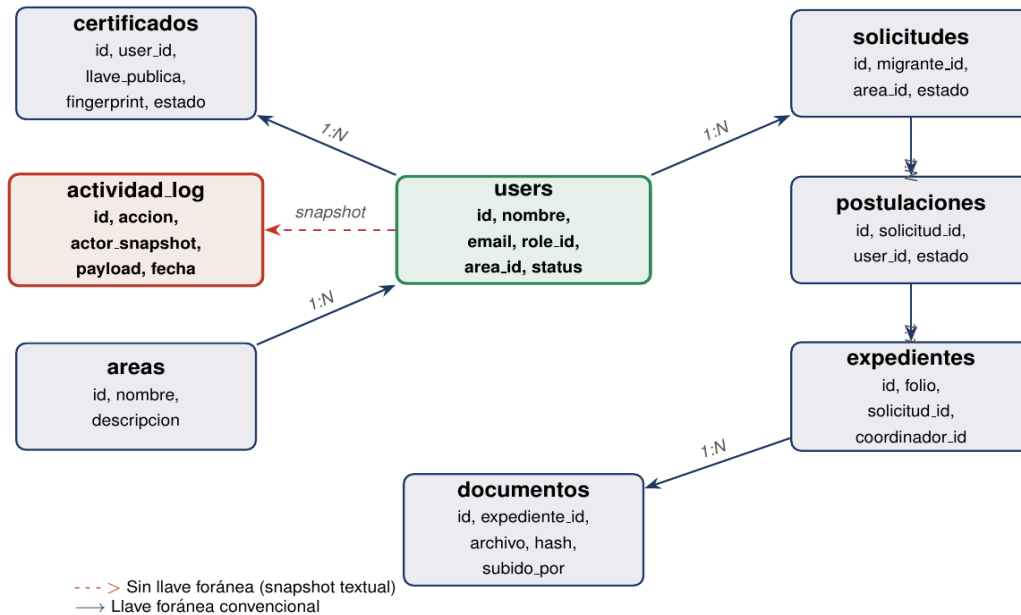


Fig. 2. Modelo entidad-relación de la base de datos (elaboración propia).

7.2. Tablas principales

Tabla	Campos clave	Relaciones
users	id, nombre, email, role_id, area_id, status	1:N → solicitudes, certificados
certificados	id, user_id, llave_publica, fingerprint, estado	N:1 → users
areas	id, nombre, descripción	1:N → users, solicitudes
solicitudes	id, migrante_id, area_id, descripción, estado	N:1 → users, areas
postulaciones	id, solicitud_id, user_id, estado	N:1 → solicitudes, users
expedientes	id, folio, solicitud_id, coordinador_id	N:1 → solicitudes, users

Tabla	Campos clave	Relaciones
documentos	id, expediente_id, archivo, hash, subido_por, sello_integridad, sellado_at, firmado_at, visible_migrante	N:1 → expedientes, users
firmas	id, documento_id, coordinador_id, firma_base64, firmado_at, challenge_hash	N:1 → documentos, users (v1.1.0)
solicitudes_rectificacion	id, migrante_id, operativo_id, documento_id, status, descripcion	N:1 → users, documentos (v1.1.0)
actividad_log	id, accion, actor_snapshot, payload, ip, fecha	Sin FK — inmutable por diseño

7.3. Columnas nuevas en v1.1.0 sobre la tabla documentos

Columna	Tipo	Propósito
sello_integridad	TEXT	HMAC-SHA256 calculado al sellar (NULL si no sellado)
sellado_at	TIMESTAMP	Fecha de aplicación del sello
firmado_at	TIMESTAMP	Fecha en que el Coordinador firmó digitalmente el documento
visible_migrante	BOOLEAN	false por defecto; true sólo tras firma del Coordinador

7.4. Migraciones nuevas en v1.1.0

- 2026_06_04_000001_add_sello_integridad_to_documentos.php — añade sello_integridad y sellado_at.
- 2026_06_04_000002_add_visible_migrante_to_documentos.php — añade el flag visible_migrante.
- 2026_06_04_000003_add_firmado_at_to_documentos.php — añade timestamp firmado_at.
- 2026_06_04_000004_create_solicitudes_rectificacion_table.php — tabla del flujo ARCO.

- 2026_06_04_000005_create_firmas_table.php — tabla de firmas digitales (v1.1.0).
- 2026_06_04_000006_add_voluntario_role.php — semilla del rol nivel 4.

7.5. Decisión: bitácora sin llave foránea

La tabla actividad_log no usa llave foránea hacia users. En su lugar, almacena un snapshot textual del actor (nombre, matrícula, rol) en el momento de la acción. Esta decisión se tomó deliberadamente para que la bitácora permanezca útil y legible incluso si el usuario es eliminado posteriormente, cumpliendo simultáneamente con el Derecho ARCO de Cancelación y con la necesidad operativa de conservar evidencia para auditoría. En v1.1.0 la bitácora también registra las descargas de archivos por parte del staff con un prefijo del hash del documento descargado, para trazabilidad adicional.

8. API / Interfaces

8.1. Rutas principales (Etapa 1)

Todas las rutas POST/PUT/DELETE requieren token CSRF (manejado por Blade). Las rutas autenticadas usan el middleware web + auth + CheckStatus.

Método	Ruta	Controlador@acción	Propósito
GET	/	WelcomeController@index	Pantalla de bienvenida
GET	/login	AuthController@showLogin	Formulario de login
POST	/login	AuthController@login	Procesar credenciales
POST	/logout	AuthController@logout	Cerrar sesión
GET	/register/colaborador	RegisterController@showColaborador	Form. registro interno
POST	/register/colaborador	RegisterController@storeColaborador	Crear cuenta interna
GET	/register/migrante	RegisterController@showMigrante	Form. entrevista migrante

Método	Ruta	Controlador@acción	Propósito
POST	/register/migrante	RegisterController@storeMigrante	Crear cuenta migrante
GET	/dashboard	DashboardController@index	Vista por rol
GET	/accesos	AccessController@index	Bandeja de Accesos
POST	/accesos/{user}/aprobar	AccessController@aprobar	Aprobar identidad
POST	/accesos/{user}/suspender	AccessController@suspender	Suspender cuenta
POST	/accesos/{user}/revocar	AccessController@revocar	Revocar identidad
POST	/accesos/{user}/restaurar	AccessController@restaurar	Restaurar cuenta
POST	/expedientes/{folio}/docs	DocumentoController@store	Subir documento
PUT	/documentos/{id}	DocumentoController@update	Editar (requiere PEM)
DELETE	/documentos/{id}	DocumentoController@destroy	Eliminar (requiere PEM)

8.2. Rutas nuevas en v1.1.0

Método	Ruta	Controlador@acción	Propósito
GET	/firmas/{documento}/challenge	FirmaController@challenge	Generar nonce para firma
POST	/firmas/{documento}/store	FirmaController@store	Verificar firma RSA
POST	/documentos-identidad	DocumentoIdentidadController@store	Cargar documento de identidad
POST	/documentos-identidad/{doc}/sellar	DocumentoIdentidadController@sellar	Aplicar HMAC-SHA256

Método	Ruta	Controlador@acción	Propósito
GET	/documentos-identidad/{doc}/download	DocumentIdentidadController@download	Descargar y registrar en log
DELETE	/documentos-identidad/{doc}	DocumentIdentidadController@destroy	Eliminar doc. de identidad
GET	/admin/archivos	ArchivosMigrantesController@index	Galería de migrantes
GET	/admin/archivos/{migrante}	ArchivosMigrantesController@show	Detalle por migrante
GET	/admin/archivos/doc/{doc}/verificar	ArchivosMigrantesController@verificar	Verif. integridad desde disco (JSON)
POST	/rectificaciones	RectificacionController@store	Iniciar solicitud ARCO
POST	/rectificaciones/{r}/tomar	RectificacionController@tomar	Operativo toma tarea
POST	/rectificaciones/{r}/propuesta	RectificacionController@subirPropuesta	Subir versión corregida
POST	/rectificaciones/{r}/aprobar	RectificacionController@aprobar	Coordinador firma y aprueba
GET	/coordinador/nuevo-caso	CoordinadorController@nuevoCasoForm	Form. nuevo caso
POST	/coordinador/nuevo-caso	CoordinadorController@nuevoCaso	Crear caso directamente

8.3. Ejemplo de petición/respuesta — firma challenge-response

Solicitud del challenge (GET):

```
GET /firmas/42/challenge HTTP/1.1
Host: casamonarca.local
Cookie: laravel_session=...

--- Respuesta ---
HTTP/1.1 200 OK
Content-Type: application/json

{
  "challenge": "8e3a5b9c2f1d4...3a7b8c9d",
  "document_id": 42,
  "expires_in": 300
}
```

Envío de la firma (POST):

```
POST /firmas/42/store HTTP/1.1
Host: casamonarca.local
Content-Type: application/json
X-CSRF-TOKEN: ...
Cookie: laravel_session=...

{
  "signature": "ZGVtb19zaWduYXR1cmVfYmFzZTY0...",
  "document_id": 42
}

--- Respuesta exitosa ---
HTTP/1.1 200 OK
{ "status": "ok", "firmado_at": "2026-06-09T14:32:18Z" }

--- Respuesta de firma inválida ---
HTTP/1.1 403 Forbidden
{ "status": "error", "message": "Firma invalida" }
```

9. Flujos y lógica de negocio

9.1. Emisión de certificado al aprobar Coordinador

La Fig. 3 ilustra el flujo. El pseudocódigo simplificado:

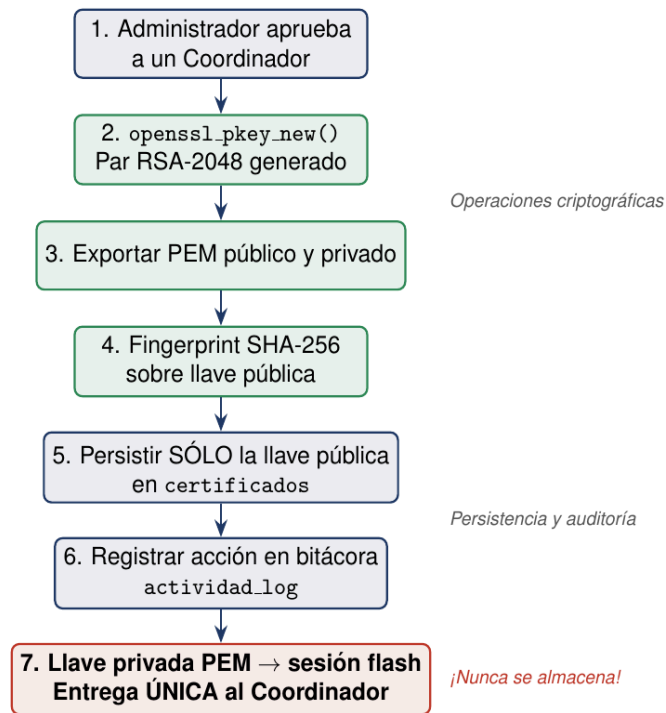


Fig. 3. Flujo de emisión de certificado RSA-2048 (elaboración propia).

```
function aprobarCoordinador($userId) {
    DB::transaction(function() use ($userId) {
        $user = User::findOrFail($userId);
        if ($user->role_id != Rol::COORDINADOR) return;

        // 1. Generar par RSA-2048
        $config = ["digest_alg" => "sha256",
                  "private_key_bits" => 2048,
                  "private_key_type" => OPENSSL_KEYTYPE_RSA];
        $key = openssl_pkey_new($config);

        // 2. Exportar partes publica y privada
        openssl_pkey_export($key, $privatePem);
        $publicPem = openssl_pkey_get_details($key)["key"];

        // 3. Calcular fingerprint SHA-256 de la publica
        $fingerprint = hash("sha256", $publicPem);

        // 4. Persistir SOLO la llave publica
        Certificado::create([
            "user_id" => $user->id,
```

```

        "llave_publica" => $publicPem,
        "fingerprint" => $fingerprint,
        "estado" => "activo",
        "emitido_en" => now(),
    ]));

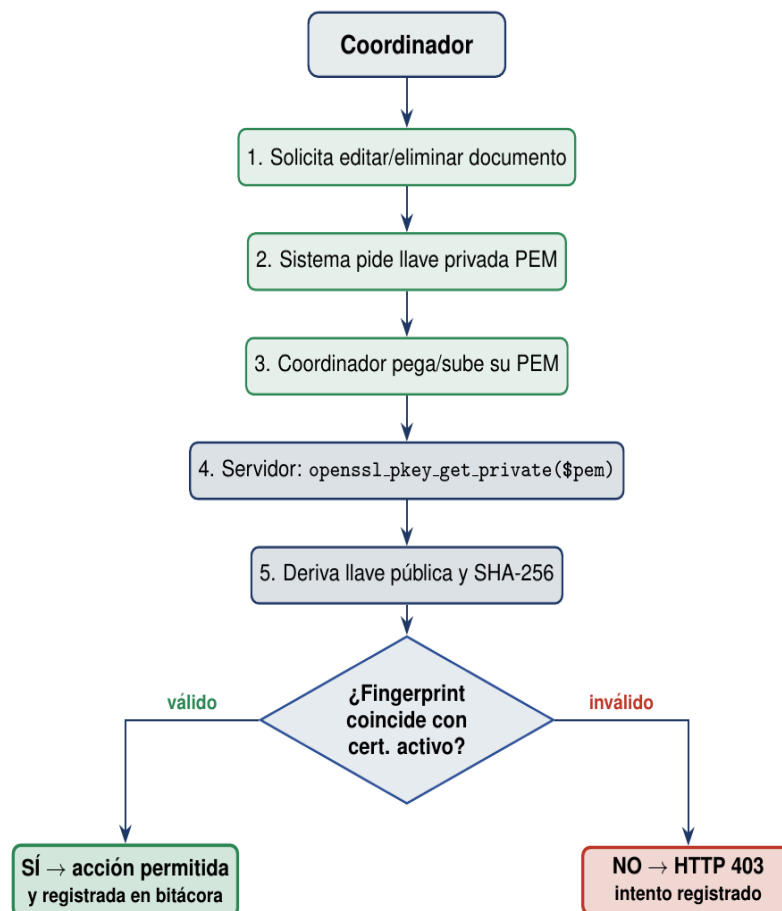
    // 5. Cambiar estado del usuario
    $user->update(["status" => "alta"]);

    // 6. Bitacora
    ActividadLog::registrar("EMITIR_CERTIFICADO", $user);

    // 7. Llave privada -> sesion flash, UNA SOLA VEZ
    session()->flash("pem_privada", $privatePem);
    });
}

```

9.2. Verificación PEM para acciones sensibles (Etapa 1)



*La comparación usa `hash_equals()` para evitar timing attacks.
El PEM ingresado nunca se persiste en servidor.*

Fig. 4. Flujo de verificación PEM (elaboración propia).

```

function verificarPem(string $pem, User $user): bool {
    // 1. Cargar certificado activo del usuario
    $cert = $user->certificadoActivo();
    if (!$cert) return false;

    // 2. Cargar llave privada desde PEM ingresado
    $privateKey = openssl_pkey_get_private($pem);
    if (!$privateKey) return false;

    // 3. Derivar la llave publica correspondiente
    $publicDerivada = openssl_pkey_get_details($privateKey)["key"];

    // 4. Comparar fingerprints en tiempo constante
    $fingerprintDerivado = hash("sha256", $publicDerivada);
    return hash_equals($cert->fingerprint, $fingerprintDerivado);
}

```

9.3. Cálculo de hash de documento al subir

```

function guardarDocumento($file, $expedienteId, $userId) {
    $path = $file->store("documentos");
    $absolutePath = Storage::path($path);
    $hash = hash_file("sha256", $absolutePath);

    Documento::create([
        "expediente_id" => $expedienteId,
        "archivo" => $path,
        "hash" => $hash,
        "subido_por" => $userId,
        "subido_en" => now(),
        "visible_migrante" => false,    // v1.1.0
    ]);
}

```

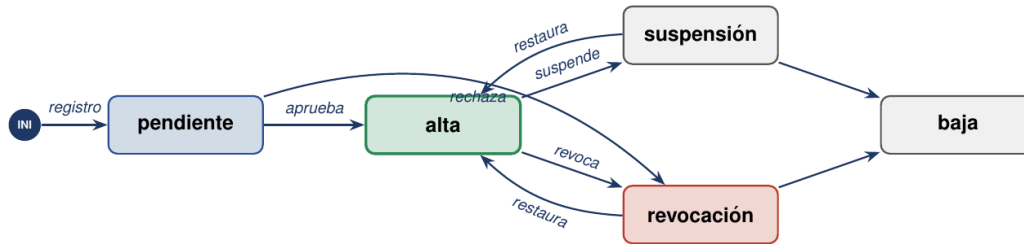
9.4. Generación de folio CM-AAAA-XXXX

```

function nuevoFolio(): string {
    $year = date("Y");
    $last = Expediente::where("folio", "like", "CM-{$year}-%")
        ->orderByDesc("id")->first();
    $next = $last ? ((int) substr($last->folio, -4)) + 1 : 1;
    return sprintf("CM-%s-%04d", $year, $next);
}

```

9.5. Ciclo de vida de una identidad



- El estado **alta** es el único que permite ingresar al sistema.
- El middleware **CheckStatus** bloquea cualquier otro estado.
- La transición a **revocación** marca todos los certificados activos del usuario como revocados.

Fig. 5. Ciclo de vida de una identidad digital (elaboración propia).

9.6. Firma digital challenge-response (v1.1.0)

El protocolo se ejecuta en dos peticiones HTTP coordinadas entre cliente y servidor. La llave privada del Coordinador nunca abandona el navegador.

Cliente — Alpine.js + Web Crypto API:

```
async function firmarDocumento(pemString, documentoId) {
  // 1. Pedir challenge al servidor
  const r = await fetch(`/firmas/${documentoId}/challenge`);
  const { challenge } = await r.json();

  // 2. Importar llave privada desde PEM
  const pemBody = pemString.replace(/-----[^\-]+-----/g, "").trim();
  const der = Uint8Array.from(atob(pemBody), c => c.charCodeAt(0));
  const key = await crypto.subtle.importKey(
    "pkcs8", der.buffer,
    { name: "RSASSA-PKCS1-v1_5", hash: "SHA-256" },
    false, ["sign"]
  );

  // 3. Firmar el challenge
  const enc = new TextEncoder().encode(challenge);
  const sigBuf = await crypto.subtle.sign(
    "RSASSA-PKCS1-v1_5", key, enc
  );
  const signature = btoa(String.fromCharCode(...new Uint8Array(sigBuf)));

  // 4. Enviar firma al servidor
  return fetch(`/firmas/${documentoId}/store`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "X-CSRF-TOKEN": csrf },
    body: JSON.stringify({ signature, document_id: documentoId })
  });
}
```

Servidor — FirmaController@store:

```
public function store(Request $r, Documento $documento) {
    $challenge = session("challenge_doc_" . $documento->id);
    abort_unless($challenge, 419, "Challenge expirado");

    $cert = auth()->user()->certificadoActivo();
    $signature = base64_decode($r->input("signature"));
    $pubKey = openssl_pkey_get_public($cert->llave_publica);

    $ok = openssl_verify($challenge, $signature, $pubKey, OPENSSL_ALGO_SHA256);
    abort_unless($ok === 1, 403, "Firma invalida");

    Firma::create([
        "documento_id" => $documento->id,
        "coordinador_id" => auth()->id(),
        "firma_base64" => $r->input("signature"),
        "firmado_at" => now(),
        "challenge_hash" => hash("sha256", $challenge),
    ]);

    $documento->update(["firmado_at" => now(), "visible_migrante" => true]);
    session()->forget("challenge_doc_" . $documento->id);
    ActividadLog::registrar("FIRMAR_DOCUMENTO", auth()->user(),
        ["doc_id" => $documento->id, "fingerprint" => $cert->fingerprint]);
    return back()->with("status", "Documento firmado");}
```

9.7. Sellado HMAC-SHA256 de documento de identidad (v1.1.0)

```
public function sellar(Request $r, Documento $documento) {
    abort_unless($documento->migrante_id === auth()->id(), 403);

    // sello = HMAC-SHA256(hash_archivo, APP_KEY)
    $sello = hash_hmac("sha256",
        $documento->hash,
        config("app.key"));

    $documento->update([
        "sello_integridad" => $sello,
        "sellado_at" => now(),
    ]);

    ActividadLog::registrar("SELLAR_DOCUMENTO_IDENTIDAD",
        auth()->user(),
        ["doc_id" => $documento->id, "sello_prefix" => substr($sello,0,16)]);

    return redirect()->route("portal.documentos")
        ->with("status", "Sello aplicado");
}

// Verificacion en Documento::selladoEsValido()
public function selladoEsValido(): bool {
    if (!$this->sello_integridad) return false;
    $hashActual = hash_file("sha256", Storage::path($this->archivo));
    if (!hash_equals($this->hash, $hashActual)) return false;
    $selloRecalc = hash_hmac("sha256", $hashActual, config("app.key"));
    return hash_equals($this->sello_integridad, $selloRecalc);
}
```

9.8. Máquina de estados del flujo ARCO de Rectificación (v1.1.0)

Estado	Quién actúa	Acción que ejecuta
pendiente_correccion	Migrante	store() — describe la corrección desde su portal
en_proceso	Operativo	tomar() — toma la solicitud y queda asignada
esperando_aprobacion	Operativo	subirPropuesta() — sube versión corregida
aprobado	Coordinador	aprobar() — firma con .pem, elimina original, activa corregido

Al aprobar, el Coordinador ejecuta el mismo protocolo challenge-response que en 9.6. Ambas versiones del documento — la eliminada y la nueva — quedan registradas en `actividad_log` con el fingerprint del certificado que autorizó el cambio.

10. Ejemplos de uso

10.1. Ejecutar pruebas

```
# Suite completa
php artisan test

# Solo un archivo especifico
./vendor/bin/pest tests/Feature/FirmaTest.php

# Con cobertura (requiere xdebug)
./vendor/bin/pest --coverage

# Filtrar por nombre del test
./vendor/bin/pest --filter="puedesFirmarConPemValido"
```

10.2. Crear usuario administrador desde Tinker

```
php artisan tinker

>>> User::create([
...     "nombre" => "Admin Bootstrap",
...     "matricula" => "ADMIN001",
...     "correo" => "admin@casamonarca.org",
...     "password" => bcrypt("CambiarEnPrimerLogin!"),
...     "role_id" => 1,
...     "status" => "alta"
... ]);
```

10.3. Consultar bitácora

```
>>> ActividadLog::where("accion", "EMITIR_CERTIFICADO")
...   ->orderByDesc("fecha")
...   ->take(10)
...   ->get()
...   ->pluck("actor_snapshot", "fecha");

# Filtrar acciones de firma digital (v1.1.0)
>>> ActividadLog::where("accion", "FIRMAR_DOCUMENTO")
...   ->whereDate("fecha", today())
...   ->count();
```

10.4. Verificar integridad de un documento desde código

```
>>> $doc = Documento::find(42);
>>> $doc->selladoEsValido();
// true | false

>>> $doc->sellado_at
// "2026-06-04 18:25:12"

>>> substr($doc->sello_integridad, 0, 16)
// "a3f1b8d2e7c4..."
```

10.5. Aprobar identidad desde la interfaz

La Fig. 6 muestra la Bandeja de Accesos donde el Administrador aprueba identidades.

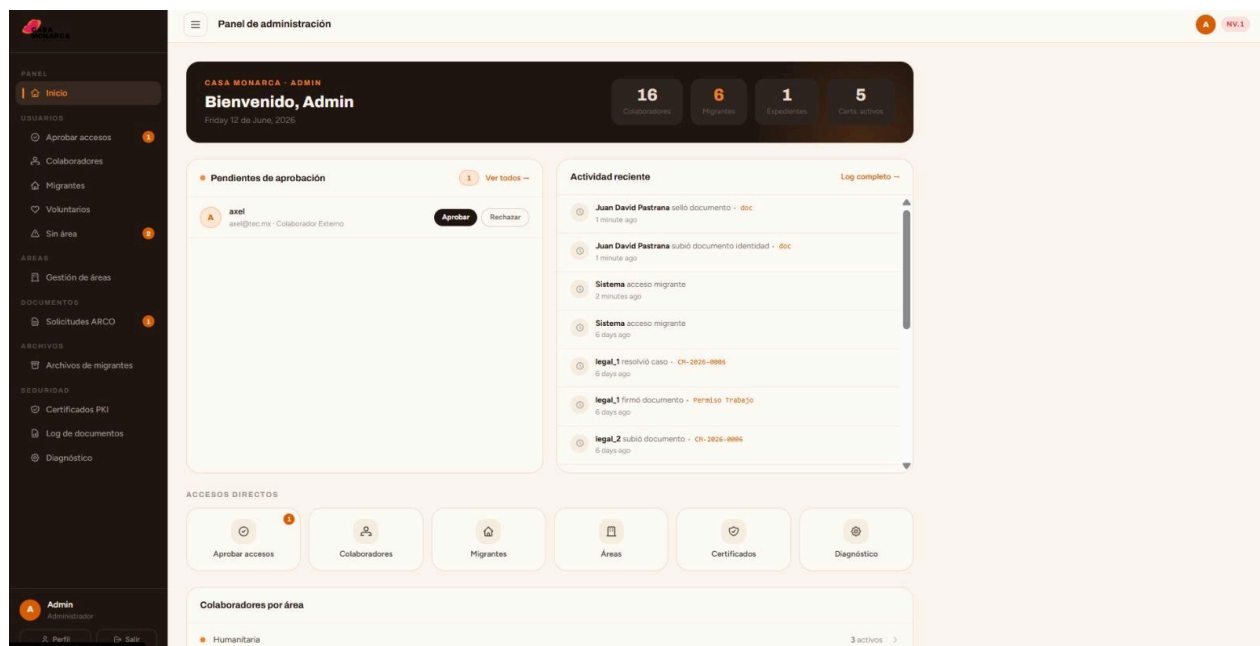


Fig. 6. Bandeja de Accesos — vista del Administrador (elaboración propia).

10.6. Crear caso directo desde Coordinador (v1.1.0)

```
# Como Coordinador autenticado
GET /coordinador/nuevo-caso

# Formulario: seleccionar migrante, área, descripción inicial
POST /coordinador/nuevo-caso
- migrante_id
- area_id
- descripcion

# El sistema crea Expediente con folio CM-AAAA-XXXX
# y queda listo para asignación de Operativo.
```

11. Manejo de errores

11.1. Estrategia general

El sistema utiliza el manejador de excepciones de Laravel (App/Exceptions/Handler.php) para capturar errores no manejados. Los errores 4xx y 5xx redirigen a vistas Blade dedicadas. Las validaciones de entrada se manejan mediante FormRequests, devolviendo HTTP 422 con detalle de campos inválidos. Las acciones sensibles que fallan por PEM inválido devuelven HTTP 403 y dejan registro en bitácora. En v1.1.0, el manejo se extiende: el endpoint de firma devuelve HTTP 419 cuando el challenge ha expirado (la sesión cambió), y la verificación de integridad desde disco devuelve un JSON estructurado para consumo AJAX.

11.2. Mensajes comunes

Código	Causa	Acción
HTTP 401	Sesión expirada o no autenticada	Redirigir a /login
HTTP 403	PEM inválido, firma inválida, o sin permisos	Mostrar mensaje específico; registrar intento
HTTP 404	Recurso inexistente	Mostrar página de no encontrado
HTTP 419	Token CSRF o challenge expirado	Recargar y reintentar
HTTP 422	Validación fallida	Mostrar errores junto a los campos
HTTP 500	Error de servidor	Registrar en storage/logs y mostrar error genérico

11.3. Registro (logs)

Los logs técnicos se persisten en storage/logs/laravel.log con rotación diaria. El nivel de log por defecto en producción es notice. La bitácora de auditoría (tabla actividad_log) es independiente del log técnico y constituye el registro inmutable de las acciones de negocio. En v1.1.0 se añade el registro de descargas de archivos por parte del staff, con un prefijo del hash del documento descargado para trazabilidad adicional sin exponer el hash completo.

12. Pruebas

12.1. Tipos de tests

- **Pruebas unitarias (tests/Unit):** aíslan servicios criptográficos y de hashing sin acceso a base.
- **Pruebas funcionales (tests/Feature):** ejercen los flujos completos a través de las rutas HTTP, contra una base SQLite en memoria.
- **Pruebas de integración con Eloquent:** validan persistencia, scopes y relaciones.

12.2. Cómo ejecutarlos

```
# Suite completa
php artisan test

# Con Pest
./vendor/bin/pest

# Filtrar por nombre
./vendor/bin/pest --filter="emite_certificado"

# Ejecutar un archivo específico
php artisan test tests/Feature/FirmaTest.php
```

12.3. Archivos de test nuevos en v1.1.0

Archivo	Qué válida
tests/Feature/FirmaTest.php	Challenge-response con .pem válido e inválido; verificación de visible_migrante
tests/Feature/SelladoDocIdentidadTest.php	store + sellar + integridad; recalculación de hash desde disco
tests/Feature/RectificacionTest.php	Flujo completo ARCO: solicitar → tomar → subirPropuesta → aprobar
tests/Feature/ArchivosMigrantesTest.php	Vista /admin/archivos; endpoint de verificación de integridad
tests/Feature/VoluntarioTest.php	Permisos de nivel 4: puede crear caso, no puede editar ni eliminar

Tabla 4. Nuevos archivos de test en v1.1.0 (elaboración propia).

La matriz completa de casos validados va de TC-01 a TC-22 (15 heredados de Etapa 1 + 7 nuevos en v1.1.0) y se documenta en el Reporte Técnico v1.1.0, sección 4.4.

13. Despliegue

13.1. Cómo hacerlo — Docker (recomendado en v1.1.0)

1. Asegurar que Docker y Docker Compose están instalados en el servidor destino.
2. Clonar el repositorio: git clone <https://github.com/A01751471/casa-monarca.git>
3. Copiar .env.example a .env y configurar DB_CONNECTION=mysql, credenciales reales, APP_ENV=production, APP_DEBUG=false.
4. docker compose up -d --build
5. docker compose exec app composer install --no-dev --optimize-autoloader
6. docker compose exec app php artisan key:generate
7. docker compose exec app php artisan migrate --seed --force
8. docker compose exec app npm ci && docker compose exec app npm run build
9. Configurar el reverse proxy (Nginx) con HTTPS apuntando al servicio app en el puerto 8000.
10. Activar tareas programadas (cron del host o servicio dedicado): * * * * * php artisan schedule:run

13.2. Cómo hacerlo — hosting tradicional

1. Configurar servidor con PHP 8.3, Composer, Node.js y MySQL 8.
2. Clonar el repositorio en el servidor.
3. Ejecutar `composer install --no-dev --optimize-autoloader`.
4. Ejecutar `npm ci && npm run build`.
5. Configurar `.env` con `APP_ENV=production`, `APP_DEBUG=false` y credenciales reales.
6. Ejecutar `php artisan key:generate`, `php artisan migrate --force`, y los caches `config:cache`, `route:cache`, `view:cache`.
7. Configurar el servidor web (Nginx o Apache) apuntando al directorio `public/`.
8. Configurar HTTPS con certificado válido (Let's Encrypt o equivalente).
9. Configurar tareas programadas con cron para `php artisan schedule:run`.

13.3. Entornos

- **local:** desarrollo con `php artisan serve` y `npm run dev`. Base SQLite (v1.1.0).
- **staging:** pruebas de integración antes de producción. Configuración idéntica a producción pero con datos no sensibles.
- **production:** ambiente real con Docker, MySQL 8 y HTTPS estricto.

13.4. Consideraciones

- Validar que el plan de hosting permita ejecutar Composer y comandos Artisan. Hosting compartido suele tener limitaciones.
- Configurar cabeceras de seguridad HSTS, CSP, X-Frame-Options en el reverse proxy.
- Rotación periódica de secretos (cookies, sesiones), pero NO de `APP_KEY` sin re-sellar documentos de identidad existentes.
- Activar respaldos automáticos de la base de datos antes del go-live.
- Monitoreo de logs en `storage/logs/laravel.log` con alertas para errores 5xx.

14. Seguridad

14.1. Autenticación

Autenticación por correo y contraseña gestionada por Laravel Breeze. Las contraseñas se almacenan con bcrypt y costo configurable. La sesión usa cookies HttpOnly y SameSite=Lax. El middleware CheckStatus bloquea cualquier intento de acceso de cuentas en estado distinto a "alta" y redirige a una pantalla informativa.

Para acciones sensibles (firma de documentos, eliminación, modificación con efecto irreversible), se requiere prueba criptográfica adicional: en Etapa 1, mediante verificación de posesión de PEM; en v1.1.0, mediante firma digital challenge-response RSA-2048 ejecutada en el navegador. La autenticación multifactor TOTP queda como pendiente prioritario para versiones siguientes.

14.2. Manejo de datos sensibles

- La llave privada PEM de un Coordinador NUNCA se persiste en servidor; sólo se entrega una vez en sesión flash al aprobar.
- Sólo la llave pública y su fingerprint SHA-256 se guardan en la tabla certificados.
- Los archivos de documento se guardan en storage privado (storage/expedientes/ y storage/identidad/), accesibles sólo a través del controlador autenticado.
- Las contraseñas de usuarios se almacenan exclusivamente como hash bcrypt.
- En v1.1.0, los documentos de identidad de migrantes quedan sellados con HMAC-SHA256 usando APP_KEY como llave; cualquier alteración del archivo en disco es detectable.
- La llave privada del Coordinador, durante el proceso de firma, se mantiene exclusivamente en memoria del navegador y se descarta tras la operación.

14.3. Buenas prácticas aplicadas

- CSRF tokens obligatorios en todos los formularios y peticiones POST/PUT/DELETE.
- Validación de entrada en FormRequests dedicados.
- Principio de mínimo privilegio aplicado mediante Gates y políticas; en v1.1.0 se añade el rol Voluntario con sólo permisos de creación.
- Auditoría inmutable para detección posterior de incidentes; incluye registro de descargas en v1.1.0.
- Uso de hash_equals() para comparar fingerprints y sellos en tiempo constante, evitando timing attacks.

- Challenge fresco por sesión y por documento en el flujo de firma, evitando ataques por replay.
- Visibilidad de documentos diferida hasta firma del Coordinador, previniendo difusión accidental.

15. Guía de contribución

15.1. Convenciones de código

- PHP: PSR-12, ejecutado automáticamente con Laravel Pint.
- Nombres de clases: PascalCase. Métodos y variables: camelCase. Constantes: UPPER_SNAKE_CASE.
- Vistas Blade: kebab-case en nombres de archivo.
- JavaScript / componentes Alpine.js: kebab-case en archivos, camelCase en funciones.
- Commits: mensajes en español, modo imperativo: "agrega revocación de certificados", "corrige verificación HMAC".

15.2. Flujo de trabajo (Git)

1. Crear branch a partir de main con nombre descriptivo: feat/firma-correos, fix/sello-hmac, docs/api-firma.
2. Desarrollar y agregar tests Pest cubriendo el cambio.
3. Ejecutar pint y la suite de pruebas localmente antes de hacer push.
4. Abrir Pull Request hacia main describiendo el cambio y referenciando el TC-XX afectado si aplica.
5. Revisión por al menos un miembro del equipo; mergear con squash una vez aprobado.
6. Actualizar CHANGELOG.md con la nueva entrada en la sección [Unreleased] hasta el corte de versión.

15.3. Cómo agregar funcionalidades

1. Crear migración si requiere cambios de esquema.
2. Crear o actualizar el modelo Eloquent correspondiente.
3. Implementar la lógica de negocio en un servicio dedicado (app/Services), no en el controlador.
4. Agregar las rutas en routes/web.php con middlewares apropiados.
5. Crear o actualizar el FormRequest de validación.
6. Crear el controlador y la vista Blade.
7. Agregar el Gate o Policy de autorización si aplica.
8. Escribir tests Feature y Unit.

9. Actualizar la bitácora con la nueva acción si aplica (ActividadLog::registrar).
10. Documentar el cambio en CHANGELOG.md y, si toca arquitectura, en este documento.

15.4. Reglas especiales para código criptográfico (v1.1.0)

- Antes de modificar FirmaController, DocumentIdentidadController o RectificacionController, leer la sección 9 completa de este documento.
- No tocar el modelo actividad_log ni la tabla certificados sin entender su semántica de inmutabilidad.
- Cualquier cambio en el protocolo de firma debe mantener pasando los tests TC-16 a TC-22.
- La APP_KEY no debe rotarse sin un procedimiento de re-sellado para los documentos de identidad existentes.
- Las comparaciones de hashes o firmas deben usar hash_equals() para evitar timing attacks; nunca usar == ni ===

16. Problemas conocidos

16.1. Bugs

Al cierre de la versión 1.1.0, no se documentan bugs bloqueantes. Los issues identificados durante el desarrollo y resueltos están registrados en el CHANGELOG. Cualquier defecto encontrado posteriormente debe registrarse en GitHub Issues del repositorio.

16.2. Limitaciones

- Sistema entregado en estado piloto sobre desarrollo local; no desplegado en producción al cierre de v1.1.0.
- No implementa autenticación multifactor (TOTP) — riesgo principal pendiente antes de go-live.
- No exporta certificados en formato X.509 estándar ni emite CRL conforme RFC 5280.
- La verificación externa pública de documentos por terceros (sin sesión autenticada) no está implementada (Sub-problema 4 del reto).
- No implementa cifrado de base de datos en reposo (Sub-problema 6).
- No incorpora sistema de firma de correos electrónicos (Sub-problema 2).
- La verificación de integridad en /admin/archivos es síncrona y puede tardar con volúmenes grandes; se recomienda procesamiento asíncrono para producción a escala.

- La rotación de APP_KEY invalida la verificación de sellos HMAC existentes, requiriendo un procedimiento manual de re-sellado.
- Si se contrata hosting compartido, debe verificarse compatibilidad con PHP 8.3, ejecución de comandos Artisan y Docker.

17. FAQ técnica

17.1. composer install falla en producción

Verificar que la versión de PHP en el servidor sea 8.3 o superior. Si el hosting tiene PHP 8.2, hacer downgrade del constraint en composer.json o actualizar el servidor. Confirmar que las extensiones requeridas (openssl, pdo, pdo_sqlite, pdo_mysql, mbstring, fileinfo, bcmath, ctype, json) estén habilitadas.

17.2. openssl_pkey_new() retorna false

Generalmente indica que la extensión OpenSSL no está habilitada. Verificar con "php -m | grep openssl". En Windows con XAMPP, descomentar la línea extension=openssl en php.ini. Si está habilitada y persiste el problema, revisar permisos del directorio donde Laravel intenta escribir y los logs del sistema.

17.3. Vite 8 no compila

Asegurarse de que Node.js es 18 o superior. Borrar node_modules y package-lock.json, y volver a ejecutar npm install. Verificar que postcss.config.js, tailwind.config.js y vite.config.js existan en la raíz del proyecto. Si se actualizó desde Vite 7, revisar la guía de migración oficial.

17.4. Tests fallan con error de base de datos

Confirmar que phpunit.xml apunta a una base SQLite en memoria. Borrar bootstrap/cache/*.php y reintentar. Si persiste, ejecutar php artisan migrate:fresh --env=testing. Para tests que tocan documentos sellados (TC-18 y TC-19), asegurar que APP_KEY está definida en el entorno de testing.

17.5. La firma del Coordinador siempre devuelve HTTP 403

Verificar tres puntos: (1) el .pem ingresado corresponde efectivamente al certificado activo del Coordinador autenticado — ver tabla certificados; (2) la sesión no ha expirado, por lo que el challenge sigue vigente — si el navegador cambió de pestaña por mucho tiempo o se cerró, repedir un challenge; (3) la cadena de signature en Base64 no ha sido corrompida en el transporte. Inspeccionar storage/logs/laravel.log para más detalle.

17.6. selladoEsValido() devuelve false aunque el archivo no haya cambiado

El caso más común es que APP_KEY haya cambiado tras el sellado. Una vez se rota APP_KEY, todos los sellos HMAC existentes quedan marcados como inválidos al ser recalculados. Si esto ocurre intencionalmente, ejecutar el procedimiento de re-sellado (manualmente, marcando los documentos y solicitando al migrante reconfirmar). Si fue accidental, restaurar la APP_KEY anterior desde respaldos.

17.7. Cómo regenerar el certificado de un coordinador

El Administrador suspende o revoca al usuario desde la Bandeja de Accesos, lo cual marca el certificado actual como revocado. Después restaura o aprueba nuevamente al usuario; al ser Coordinador, el sistema genera y entrega un nuevo certificado RSA-2048 con su PEM correspondiente, también una sola vez. Comunicar al Coordinador que su .pem anterior ya no es válido y entregar el nuevo en persona o por canal cifrado.

17.8. Cómo descargar dependencias en hostings restringidos

Si el hosting no permite Composer, ejecutar localmente "composer install --no-dev --optimize-autoloader" y subir el proyecto completo incluyendo la carpeta vendor/ por FTP. Para npm aplica el mismo patrón: compilar local con "npm run build" y subir public/build/. Esta operación debe repetirse cada vez que cambien las dependencias.

17.9. ¿Cómo verifico manualmente la integridad de un documento de identidad?

```
# Desde Tinker
php artisan tinker

>>> $doc = Documento::find(42);
>>> $hashEnDisco = hash_file("sha256", Storage::path($doc->archivo));
>>> $hashEnBd = $doc->hash;
>>> $sello = $doc->sello_integridad;
>>> $selloEsperado = hash_hmac("sha256", $hashEnDisco, config("app.key"));

>>> $hashEnDisco === $hashEnBd      // true → archivo no fue alterado
>>> $sello === $selloEsperado      // true → sello es valido

# Forma corta
>>> $doc->selladoEsValido()         // true / false
```

Referencias

Alpine.js. (2025). Alpine.js Documentation. <https://alpinejs.dev>

Barker, E. (2020). Recommendation for Key Management: Part 1 — General (NIST Special Publication 800-57 Part 1 Revision 5). National Institute of Standards and Technology.

Cámara de Diputados del H. Congreso de la Unión. (2025). Ley Federal de Protección de Datos Personales en Posesión de los Particulares. Diario Oficial de la Federación.

Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., y Polk, W. (2008). Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile (RFC 5280). Internet Engineering Task Force.

Docker, Inc. (2025). Docker Compose Documentation. <https://docs.docker.com/compose/>

Krawczyk, H., Bellare, M., y Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication (RFC 2104). Internet Engineering Task Force.

Laravel. (2025). Documentación oficial de Laravel 13. <https://laravel.com/docs/13.x>

MDN Web Docs. (2025). Web Crypto API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

Paar, C., y Pelzl, J. (2010). Understanding Cryptography: A Textbook for Students and Practitioners. Springer.

Pest. (2025). Pest 4 Documentation. <https://pestphp.com/docs>

Rivest, R. L., Shamir, A., y Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126.

Tailwind Labs. (2025). Tailwind CSS Documentation. <https://tailwindcss.com/docs>

The PHP Group. (2025). PHP Manual: OpenSSL extension. <https://www.php.net/manual/en/book.openssl.php>

Vite. (2025). Vite 8 Documentation. <https://vitejs.dev>